Component-and-connect patterns:

- Broker
- Model-view Controller
- Pipe and Filter
- Client-Server
- Peer-to-Peer
- Service-Oriented-Architecture
- Publish-Subscribe
- **Shared-Data**

Shared-Data Pattern

- **Context:** Various computational components need to share and manipulate large amounts of data. This data does not belong solely to any one of those components.

- **Problem:** How can systems store and manipulate persistent data that is accessed by multiple independent components?

- **Solution:** In the shared-data pattern, interaction is dominated by the exchange of persistent data between multiple *data accessors* and at least one *shared-data store*. Exchange may be initiated by the accessors or the data store. The connector type is *data reading and writing*.

Shared Data Solution - 1

- Overview: Communication between data accessors is mediated by a shared data store. Control may be initiated by the data accessors or the data store. Data is made persistent by the data store.

- **Elements**:
  - *Shared-data store*. Concerns include types of data stored, data performance-oriented properties, data distribution, and number of accessors permitted.
  - *Data accessor component*.
  - *Data reading and writing connector*.

Shared Data Example

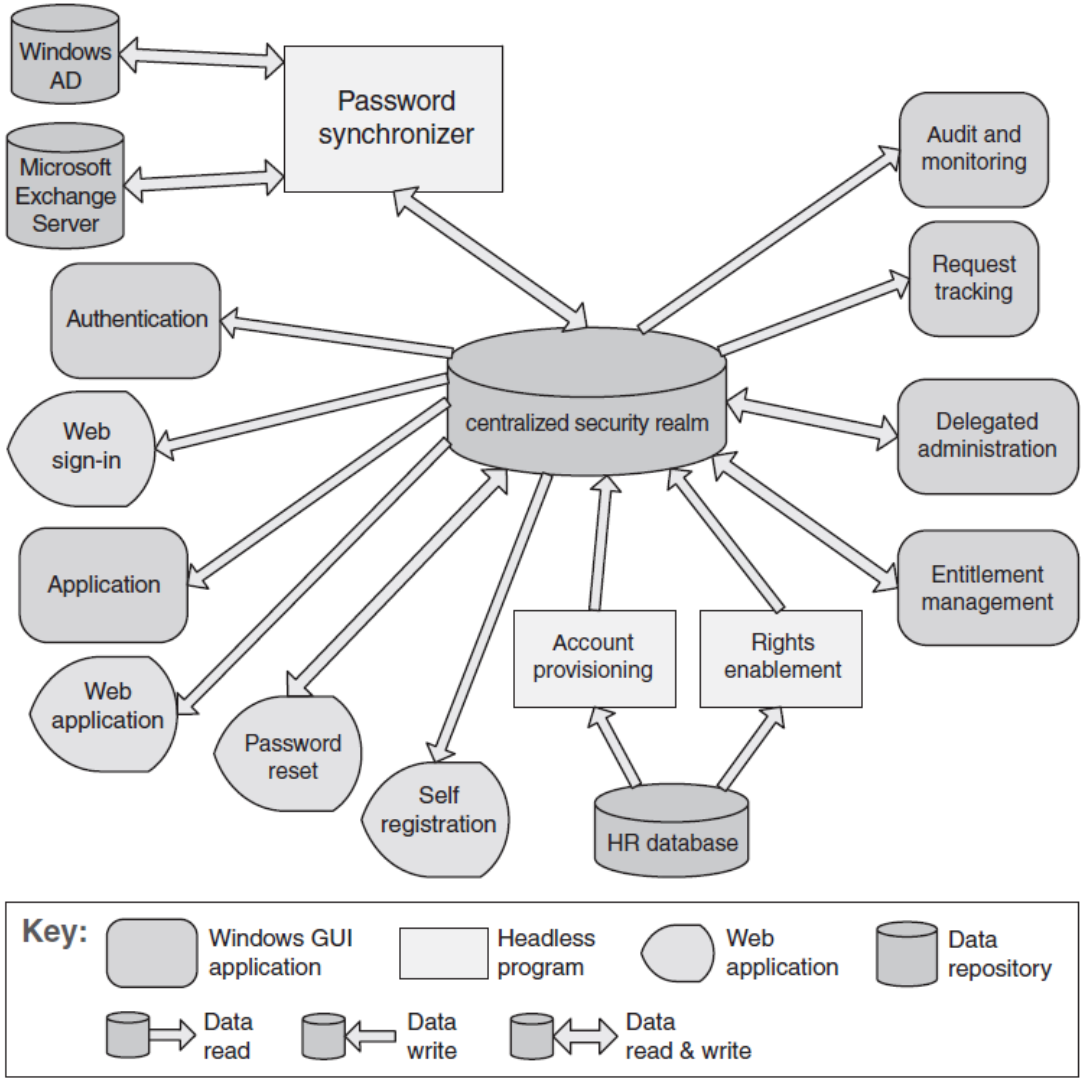

FIGURE 13.13 The shared-data diagram of an enterprise access management system

Shared Data Solution - 2

- **Relations**: *Attachment* relation determines which data accessors are connected to which data stores.

- Constraints: Data accessors interact only with the data store(s).

- **Weaknesses**:
  - The shared-data store may be a performance bottleneck.
  - The shared-data store may be a single point of failure.
  - Producers and consumers of data may be coupled; through their knowledge of the structure of the shared data.

Shared Data Solution - 3

- **Advantages:**
  - This pattern supports modifiability, as the producers do not have direct knowledge of the consumers
  - Consolidating the data in one or more locations and accessing it in a common fashion facilitates performance tuning.

Finer Points of Shared Data

- Analyses associated with this pattern usually center on qualities such as: data consistency, performance, security, privacy, availability, scalability, and compatibility with, for example, existing repositories and their data.
- When a system has more than one data store, a key architecture concern is the mapping of data and computation to the data.
  - Use of multiple stores may occur because the data is naturally, or historically, partitioned into separable stores.
- In other cases data may be replicated over several stores to improve performance or availability through redundancy.